

Tài liệu giải thích code — 2 chức năng chính

Mục lục

1. Chức năng Thêm sản phẩm
 2. Chức năng Thống kê feedback
-

1. Chức năng Thêm sản phẩm

Tổng quan luồng

```
Frontend (products.html)
→ openAddForm()           - mở form nhập liệu
→ loadCategories()         - load danh mục từ API
→ loadFixedAttrs()         - load thuộc tính cố định của danh mục
→ saveProduct()           - gửi POST /api/products

API Gateway (cổng 8888)
→ forward đến product-service (cổng 8081)

ProductController.createProduct()
→ ProductService.createProduct()
→ validate input
→ tạo Product entity
→ gán fixedAttributes (thuộc tính của danh mục)
→ gán extraAttributes (thuộc tính tự do)
→ lưu vào MySQL (product_db)
```

Frontend — `products.js`

`openAddForm()`

```
function openAddForm() {
  document.getElementById('addPanel').classList.add('open'); // hiện panel form
  loadCategories();                                           // load danh sách
  danh mục (chỉ 1 lần)
  window.scrollTo({ top: 0, behavior: 'smooth' });           // cuộn lên đầu trang
}
```

Gọi khi người dùng nhấn nút "+ **Thêm sản phẩm**". Thêm class `open` vào panel để CSS hiện panel ra.

`loadCategories()`

```
function loadCategories() {
  if (categoriesLoaded) return; // tránh gọi API nhiều lần
  api('/api/categories').then(cats => {
    const sel = document.getElementById('fCategory');
    cats.forEach(c => {
      const o = document.createElement('option');
      o.value = c.id; o.text = c.name;
      sel.appendChild(o); // thêm từng danh mục vào dropdown
    });
    categoriesLoaded = true; // đánh dấu đã load xong
  });
}
```

Gọi GET `/api/categories`, điền kết quả vào dropdown `<select id="fCategory">`. Biến `categoriesLoaded` đảm bảo chỉ gọi API **một lần duy nhất** trong suốt phiên làm việc.

loadFixedAttrs(categoryId)

```
function loadFixedAttrs(categoryId) {
  if (!categoryId) { section.style.display = 'none'; return; }
  api(`/api/categories/${categoryId}/attributes`).then(attrs => {
    if (!attrs.length) { section.style.display = 'none'; return; }
    section.style.display = 'block';
    list.innerHTML = attrs.map(a => `
      <div class="attr-row">
        <label>
          <input type="checkbox" class="fixed-attr-cb" data-id="${a.id}" .../>
          <span>${a.name}</span>
        </label>
        <input type="text" class="fixed-attr-val" ... disabled />
      </div>`).join('');
  });
}
```

Khi người dùng **chọn danh mục**, gọi GET `/api/categories/{id}/attributes` để lấy các thuộc tính cố định (ví dụ: danh mục "Laptop" có thuộc tính "RAM", "CPU"). Mỗi thuộc tính hiện ra 1 checkbox + 1 ô nhập giá trị. Ô nhập bị disabled cho đến khi tick checkbox.

toggleAttrInput(cb)

```
function toggleAttrInput(cb) {
  const inp = cb.closest('.attr-row').querySelector('.fixed-attr-val');
  inp.disabled = !cb.checked; // bật/tắt ô nhập theo checkbox
  inp.style.opacity = cb.checked ? '1' : '.4';
}
```

```
if (cb.checked) inp.focus(); // tự focus khi tick vào
}
```

Khi người dùng tick/bỏ tick thuộc tính cố định, bật/tắt ô nhập giá trị tương ứng.

addExtraRow()

```
function addExtraRow() {
  const div = document.createElement('div');
  div.className = 'extra-row';
  div.innerHTML = `
    <input type="text" placeholder="Tên thuộc tính" class="extra-attr-name" />
    <input type="text" placeholder="Giá trị" class="extra-attr-val" />
    <button onclick="this.parentElement.remove()">×</button>`;
  document.getElementById('extraAttrsList').appendChild(div);
}
```

Thêm 1 hàng nhập thuộc tính tự do (tên + giá trị). Người dùng có thể thêm bao nhiêu hàng tùy thích. Nút × xóa hàng đó.

saveProduct()

```
function saveProduct() {
  // 1. Validate
  if (!name) { showFormErr('Vui lòng nhập tên'); return; }
  if (isNaN(price) || price < 0) { showFormErr('Giá không hợp lệ'); return; }
  if (!categoryId) { showFormErr('Chọn danh mục'); return; }

  // 2. Thu thập fixedAttributes – những checkbox đã được tick
  const fixedAttributes = [];
  document.querySelectorAll('.fixed-attr-cb:checked').forEach(cb => {
    fixedAttributes.push({
      attributeId: parseInt(cb.dataset.id),
      value: cb.closest('.attr-row').querySelector('.fixed-attr-val').value.trim()
    });
  });

  // 3. Thu thập extraAttributes – các hàng tự do người dùng thêm
  const extraAttributes = [];
  document.querySelectorAll('.extra-row').forEach(row => {
    const n = row.querySelector('.extra-attr-name').value.trim();
    const v = row.querySelector('.extra-attr-val').value.trim();
    if (n) extraAttributes.push({ name: n, value: v });
  });

  // 4. Gửi POST /api/products
  api('/api/products', {
```

```

        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name, price, stockQuantity, categoryId,
fixedAttributes, extraAttributes })
    })
    .then(() => { showToast('Thêm sản phẩm thành công!', 'ok'); closeAddForm();
loadProducts(...); })
    .catch(e => showFormErr('Lỗi: ' + e.message));
}

```

Hàm chính xử lý submit form. Gom dữ liệu từ form thành 1 object JSON rồi gửi lên backend. Nếu thành công: đóng form, reload bảng sản phẩm, hiện toast. Nếu lỗi: hiện thông báo đỏ trong form.

Backend — ProductController.java

```

@PostMapping
public ResponseEntity<Product> createProduct(@RequestBody Map<String, Object>
request) {
    log.info("[ADMIN] POST /api/products name='{ }'", request.get("name"));
    Product created = productService.createProduct(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(created);
}

```

Nhận request `POST /api/products` với body JSON. Dùng `Map<String, Object>` thay vì DTO cố định vì request có thể chứa số lượng thuộc tính linh hoạt. Trả về HTTP 201 Created kèm sản phẩm vừa tạo.

Backend — ProductService.createProduct()

```

@Transactional
public Product createProduct(Map<String, Object> req) {

    // 1. Lấy và validate các field bắt buộc
    String name      = (String) req.get("name");
    Number price     = (Number) req.get("price");
    Number categoryId = (Number) req.get("categoryId");

    if (name == null || name.isBlank()) throw new IllegalArgumentException("Tên
không được trống");
    if (price == null)                  throw new IllegalArgumentException("Giá
không được trống");
    if (categoryId == null)              throw new IllegalArgumentException("Danh
mục không được trống");

    // 2. Tìm danh mục trong DB, báo lỗi nếu không tồn tại
    Category category = categoryRepository.findById(categoryId.longValue())
        .orElseThrow(() -> new IllegalArgumentException("Category không tồn
tại"));
}

```

```

// 3. Tạo Product entity
Product product = new Product();
product.setName(name.trim());
product.setPrice(new BigDecimal(price.toString()));
product.setStockQuantity(stock != null ? stock.intValue() : 0);
product.setCategory(category);

List<ProductAttribute> productAttributes = new ArrayList<>();

// 4. Xử lý fixedAttributes – thuộc tính cố định của danh mục
List<Map<String, Object>> fixedAttrs = (List<Map<String, Object>>)
req.get("fixedAttributes");
if (fixedAttrs != null) {
    for (Map<String, Object> entry : fixedAttrs) {
        Long attrId = ((Number) entry.get("attributeId")).longValue();
        String value = (String) entry.get("value");
        Attribute attr = attributeRepository.findById(attrId)
            .orElseThrow(() -> new IllegalArgumentException("Attribute
không tồn tại"));
        productAttributes.add(new ProductAttribute(product, attr, value));
    }
}

// 5. Xử lý extraAttributes – thuộc tính tự do người dùng đặt tên
List<Map<String, Object>> extraAttrs = (List<Map<String, Object>>)
req.get("extraAttributes");
if (extraAttrs != null) {
    for (Map<String, Object> entry : extraAttrs) {
        String attrName = (String) entry.get("name");
        String value = (String) entry.get("value");
        if (attrName == null || attrName.isBlank()) continue;

        // Tìm thuộc tính theo tên; nếu chưa có thì tự tạo mới
        Attribute attr = attributeRepository.findByName(attrName.trim())
            .orElseGet(() -> attributeRepository.save(new
Attribute(attrName.trim())));

        productAttributes.add(new ProductAttribute(product, attr, value));
    }
}

// 6. Gán toàn bộ thuộc tính vào product và lưu
product.setProductAttributes(productAttributes);
return productRepository.save(product);
}

```

Điểm đáng chú ý:

- `@Transactional` đảm bảo toàn bộ thao tác (tạo product + tạo attribute mới nếu có + lưu ProductAttribute) được commit hoặc rollback cùng nhau.

- `extraAttribute` dùng `findByName(...).orElseGet(...)` — tái sử dụng attribute đã tồn tại thay vì tạo trùng.

Entity — `Product.java`

```
@Entity
@Table(name = "products")
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private BigDecimal price;
    private Integer stockQuantity;

    @ManyToOne(optional = false)
    @JoinColumn(name = "category_id")
    private Category category; // N sản phẩm thuộc 1 danh mục

    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL,
        orphanRemoval = true, fetch = FetchType.EAGER)
    private List<ProductAttribute> productAttributes; // 1 sản phẩm có N thuộc tính
}
```

Điểm đáng chú ý:

- `cascade = CascadeType.ALL` — khi lưu `Product`, các `ProductAttribute` được lưu theo.
- `orphanRemoval = true` — khi xóa `Product`, tất cả `ProductAttribute` của nó cũng bị xóa.
- `FetchType.EAGER` — luôn load thuộc tính cùng sản phẩm trong 1 query.

Entity — `ProductAttribute.java`

```
@Entity
@Table(name = "product_attributes")
public class ProductAttribute {
    @JsonIgnore
    @ManyToOne @JoinColumn(name = "product_id")
    private Product product; // tham chiếu ngược về Product

    @ManyToOne @JoinColumn(name = "attribute_id")
    private Attribute attribute; // thuộc tính (ví dụ: "RAM")

    @Column(name = "attr_value")
```

```
private String value;           // giá trị cụ thể (ví dụ: "16GB")
}
```

`@JsonIgnore` trên `product` tránh vòng lặp vô hạn khi serialize JSON (Product → ProductAttribute → Product → ...).

2. Chức năng Thống kê feedback

Tổng quan luồng

Frontend (feedback-stats.html)

→ Tự động load: `init()` lấy danh sách sản phẩm để tra tên

Khi nhấn "Thống kê":

→ `doSearch()` → GET `/api/feedbacks/range?from=...&to=...`

↓

`FeedbackController.getByDateRange()`

↓

`FeedbackService.getFeedbacksByDateRange()`

↓

`enrich()` → gọi `user-service` lấy tên khách hàng

→ gọi `product-service` lấy tên thuộc tính

↓

Trả về `List<Feedback>` đã được bổ sung tên

Frontend cache toàn bộ vào `allFeedbacks[]`

→ `renderLayer1()` – tổng hợp thống kê theo sản phẩm

→ `openProduct()` – lọc từ cache, hiện tầng 2 (không gọi API mới)

→ `openFeedback()` – lọc từ cache, hiện tầng 3 (không gọi API mới)

Frontend — `feedback-stats.js`

Khởi tạo — `init()`

```
(function init() {
  const today = new Date();
  const first = new Date(today.getFullYear(), today.getMonth(), 1); // ngày 1 của
tháng hiện tại
  document.getElementById('toDate').value = fmt(today); // mặc định: hôm nay
  document.getElementById('fromDate').value = fmt(first); // mặc định: đầu tháng

  fetch('/api/products?page=0&size=500') // load tất cả sản phẩm để tra tên
    .then(r => r.json())
    .then(d => { allProducts = d.content || []; });
})();
```

Chạy ngay khi trang load. Set giá trị mặc định cho 2 ô ngày và load danh sách sản phẩm vào cache `allProducts` (dùng để tra tên sản phẩm ở tầng 1 — xem `getProductName()`).

Tầng 1 — `doSearch()` và `renderLayer1()`

```
function doSearch() {
  // Validate ngày
  if (!from || !to) { showToast('Chọn khoảng thời gian', 'err'); return; }
  if (from > to)      { showToast('Ngày đầu phải trước ngày cuối', 'err'); return; }
}

fetch(`/api/feedbacks/range?from=${from}&to=${to}`)
  .then(r => r.json())
  .then(feedbacks => {
    allFeedbacks = feedbacks; // cache toàn bộ cho tầng 2, 3
    renderLayer1(feedbacks, q, from, to);
  });
}
```

```
function renderLayer1(feedbacks, searchQ, from, to) {
  // Gom feedback theo productId
  const map = {};
  feedbacks.forEach(f => {
    if (!map[f.productId]) map[f.productId] = { productId: f.productId, feedbacks:
[] };
    map[f.productId].feedbacks.push(f);
  });

  // Tính điểm trung bình cho từng sản phẩm
  let rows = Object.values(map).map(g => {
    const total = g.feedbacks.length;
    const avg   = g.feedbacks.reduce((s, f) => s + (f.overallRating || 0), 0) /
total;
    const name  = getProductName(g.productId); // tra từ allProducts cache
    return { productId: g.productId, name, total, avg };
  });

  // Lọc theo tìm kiếm tên sản phẩm (realtime, không gọi API)
  if (searchQ) rows = rows.filter(r => r.name.toLowerCase().includes(searchQ));

  // Sắp xếp theo số phản hồi giảm dần
  rows.sort((a, b) => b.total - a.total);

  // Render bảng thống kê + 3 box tổng quan
}
```


Toàn bộ tính toán (gom nhóm, tính trung bình, lọc, sắp xếp) đều xảy ra trên frontend sau khi có dữ liệu từ API. Backend chỉ trả data thô.

Tầng 2 — `openProduct()`

```
function openProduct(productId, productName) {
  currentProduct = { id: productId, name: productName }; // lưu sản phẩm đang xem

  const list = allFeedbacks
    .filter(f => f.productId === productId) // lọc từ cache – không
    gọi API
    .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt)); // mới nhất
    lên đầu

  renderLayer2(list);
  goLayer(2); // chuyển sang tầng 2
}
```

Không có network request nào — lọc trực tiếp từ `allFeedbacks` đã cache. Đây là lý do click vào sản phẩm không tạo log ở backend.

Tầng 3 — `openFeedback()`

```
function openFeedback(feedbackId) {
  const f = allFeedbacks.find(x => x.id === feedbackId); // tìm từ cache
  if (!f) { showToast('Không tìm thấy phản hồi', 'err'); return; }

  // Render bảng đánh giá từng thuộc tính
  const attrRows = (f.attributeRatings || []).map(ar => `
    <tr>
      <td>${ar.attributeName || 'Thuộc tính #' + ar.attributeId}</td>
      <td>${starsHtml(ar.rating)} ${ar.rating}/5</td>
      <td>${ar.comment || '-'}</td>
    </tr>`).join('');

  goLayer(3);
}
```

`attributeName` đã được backend điền sẵn trong bước `enrich()` (xem phần service). Frontend chỉ việc hiển thị.

Điều hướng — `goLayer(n)` và `updateBreadcrumb(n)`

```
function goLayer(n) {
  // Ẩn tất cả layer, chỉ hiện layer thứ n
  document.querySelectorAll('.layer').forEach((el, i) => {
    el.classList.toggle('active', i + 1 === n);
  });
  updateBreadcrumb(n);
  window.scrollTo({ top: 0, behavior: 'smooth' });
}
```

3 tầng là 3 `<div class="layer">` trong HTML. `goLayer()` toggle class `active` để CSS ẩn/hiện đúng tầng.

Backend — FeedbackController.java

```
@GetMapping("/range")
public List<Feedback> getByDateRange(
    @RequestParam String from,          // ví dụ: "2026-01-01"
    @RequestParam String to,           // ví dụ: "2026-04-22"
    @RequestParam(required = false) Long productId) {
    log.info("GET /api/feedbacks/range from={} to={} productId={}", from, to,
productId);
    return feedbackService.getFeedbacksByDateRange(from, to, productId);
}
```

`productId` là optional — không truyền thì lấy tất cả sản phẩm, có truyền thì lọc theo sản phẩm đó.

Backend — FeedbackService.java

`getFeedbacksByDateRange()`

```

public List<Feedback> getFeedbacksByDateRange(String fromDate, String toDate, Long
productId) {
    // Parse chuỗi "2026-01-01" thành LocalDateTime
    LocalDateTime from = LocalDate.parse(fromDate).atStartOfDay();           // 2026-
01-01T00:00:00
    LocalDateTime to    = LocalDate.parse(toDate).atTime(LocalTime.MAX);     // 2026-
04-22T23:59:59.999999999

    List<Feedback> list = productId != null
        ? feedbackRepository.findByProductIdAndDateRange(productId, from, to)
    // lọc theo sản phẩm
        : feedbackRepository.findByDateRange(from, to);
    // lấy tất cả

    return enrich(list); // bổ sung tên trước khi trả về
}

```

enrich()

```

private List<Feedback> enrich(List<Feedback> list) {
    Map<Long, String> customerCache = new HashMap<>(); // tránh gọi user-service
trùng
    Map<Long, String> attributeCache = new HashMap<>(); // tránh gọi product-
service trùng

    for (Feedback fb : list) {
        // Lấy tên khách hàng – nếu cùng customerId đã gọi rồi thì dùng cache
        String cname = customerCache.computeIfAbsent(fb.getCustomerId(), cid -> {
            CustomerDto c = userServiceClient.getCustomer(cid);
            return c != null ? c.getFullName() : null;
        });
        fb.setCustomerName(cname); // gán vào @Transient field

        // Lấy tên thuộc tính cho từng AttributeRating
        for (AttributeRating ar : fb.getAttributeRatings()) {
            String aname = attributeCache.computeIfAbsent(ar.getAttributeId(), aid
-> {
                AttributeDto a = productServiceClient.getAttribute(aid);
                return a != null ? a.getName() : null;
            });
            ar.setAttributeName(aname); // gán vào @Transient field
        }
    }
    return list;
}

```

Điểm đáng chú ý:

- `@Transient` trên `customerName` và `attributeName` nghĩa là 2 field này **không lưu vào DB** — chỉ tồn tại trong bộ nhớ khi trả response JSON.
- `computeIfAbsent` — nếu key đã có trong cache thì không gọi inter-service nữa. Ví dụ: 10 feedback cùng `customerId=3` thì chỉ gọi user-service **1 lần**.

Backend — `FeedbackRepository.java`

```
@Query("SELECT f FROM Feedback f WHERE f.createdAt BETWEEN :from AND :to ORDER BY f.createdAt DESC")
List<Feedback> findByDateRange(@Param("from") LocalDateTime from, @Param("to")
LocalDateTime to);

@Query("SELECT f FROM Feedback f WHERE f.productId = :productId AND f.createdAt
BETWEEN :from AND :to ORDER BY f.createdAt DESC")
List<Feedback> findByIdAndDateRange(@Param("productId") Long productId,
                                     @Param("from") LocalDateTime from,
                                     @Param("to") LocalDateTime to);
```

Dùng JPQL (`SELECT f FROM Feedback f`) thay vì SQL thuần để query qua entity. `BETWEEN :from AND :to` map với cột `createdAt` trong bảng `feedbacks`.

Entity — `Feedback.java`

```
@Entity
@Table(name = "feedbacks")
public class Feedback {
    private Long productId; // ID sản phẩm được đánh giá (không có FK –
microservice)
    private Long customerId; // ID khách hàng (không có FK – microservice)
    private String comment;
    private Integer overallRating;
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "feedback", cascade = CascadeType.ALL,
fetch = FetchType.EAGER)
    private List<AttributeRating> attributeRatings; // đánh giá từng thuộc tính

    @Transient
    private String customerName; // không lưu DB, chỉ điền khi trả response
}
```

`productId` và `customerId` lưu dạng số nguyên thay vì foreign key vì đây là **microservice** — feedback-service không có quyền truy cập trực tiếp bảng `products` hay `users`.

Helpers — `feedback-stats.js`

Hàm	Vai trò
<code>getProductName(productId)</code>	Tra tên sản phẩm từ <code>allProducts</code> cache; fallback về "Sản phẩm #id" nếu không tìm thấy
<code>starsHtml(n)</code>	Tạo HTML hiển thị sao đầy/rỗng theo điểm số (1–5)
<code>scoreClass(n)</code>	Trả về CSS class <code>score-hi</code> / <code>score-mid</code> / <code>score-lo</code> theo điểm
<code>fmtDt(iso)</code>	Format ISO datetime sang kiểu Việt Nam: <code>22/04/2026 11:15</code>
<code>esc(s)</code>	Escape HTML đặc biệt (<code><</code> , <code>></code> , <code>&</code> , <code>"</code>) để chống XSS khi render dữ liệu người dùng nhập
<code>showToast(msg, type)</code>	Hiện thông báo nhỏ góc màn hình, tự ẩn sau 3 giây